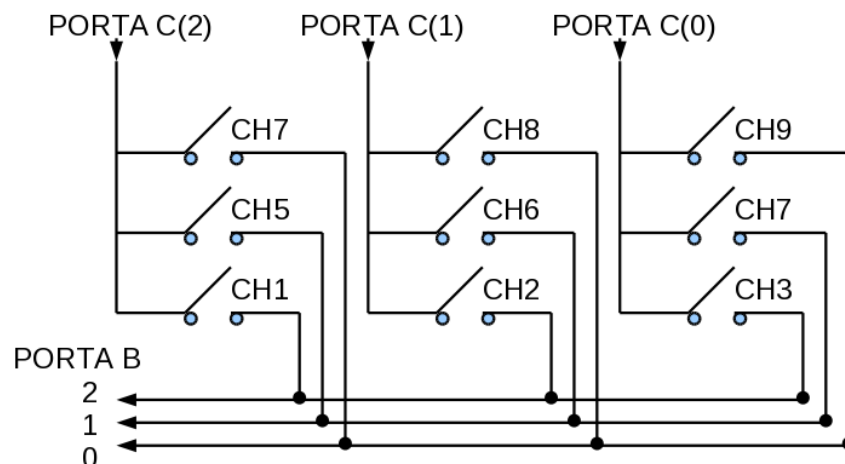


bit 7	WCOL: Write Collision Detect bit <u>In Master Transmit mode:</u> 1 = A write to the SSPBUF register was attempted while the I ² C conditions were not valid for a transmission to be started (must be cleared in software) 0 = No collision <u>In Slave Transmit mode:</u> 1 = The SSPBUF register is written while it is still transmitting the previous word (must be cleared in software) 0 = No collision <u>In Receive mode (Master or Slave modes):</u> This is a "don't care" bit.
bit 6	SSPOV: Receive Overflow Indicator bit <u>In Receive mode:</u> 1 = A byte is received while the SSPBUF register is still holding the previous byte (must be cleared in software) 0 = No overflow <u>In Transmit mode:</u> This is a "don't care" bit in Transmit mode.
bit 5	SSPEN: Master Synchronous Serial Port Enable bit 1 = Enables the serial port and configures the SDA and SCL pins as the serial port pins ⁽¹⁾ 0 = Disables serial port and configures these pins as I/O port pins ⁽¹⁾
bit 4	CKP: SCK Release Control bit <u>In Slave mode:</u> 1 = Release clock 0 = Holds clock low (clock stretch), used to ensure data setup time <u>In Master mode:</u> Unused in this mode.
bit 3-0	SSPM3:SSPM0: Master Synchronous Serial Port Mode Select bits 1111 = I ² C Slave mode, 10-bit address with Start and Stop bit interrupts enabled ⁽²⁾ 1110 = I ² C Slave mode, 7-bit address with Start and Stop bit interrupts enabled ⁽²⁾ 1011 = I ² C Firmware Controlled Master mode (slave Idle) ⁽²⁾ 1000 = I ² C Master mode, clock = $F_{OSC}/(4 * (SSPADD + 1))$ ⁽²⁾ 0111 = I ² C Slave mode, 10-bit address ⁽²⁾ 0110 = I ² C Slave mode, 7-bit address ⁽²⁾

- Note 1:** When enabled, the SDA and SCL pins must be properly configured as input or output.
- Note 2:** Bit combinations not specifically listed here are either reserved or implemented in SPI mode only.

```
SSPCON1 = 0bxx1x1000; //os bits com x não importam
```

(25 pts Questão 3) Um sistema embarcado possui as 9 chaves ligadas conforme a figura abaixo (adotar a porta C como saída e porta B como entrada). As chaves **não possuem** efeito de bouncing.



Monte uma **biblioteca** (teclado.c e teclado.h) com duas funções:

- 1) "void InicializaTeclado(void)": a função fará a inicialização do hardware conforme necessário.
- 2) "void VarreduraChaves(void)": a função fará a varredura das chaves e salvar o estado de cada

chave numa variável global-local "unsigned int Teclas".

// O programa abaixo foi feito supondo que os pinos são ligados com 1.
// Pode ser feito também com os pinos sendo ligados em zero,
// sendo necessário apenas modificar as constantes. O restante fica inalterado
unsigned int Teclas; //variável global-local

```
void InicializaTeclado(void){
    TRISB = 0x07;           //configura (210) como entrada
    TRISC = 0x00;           //configura (210) como saída
}

void VarreduraChaves(void){
    unsigned char i,j;

    for(i = 0; i < 3; i++){
        PORTC = 0x00;        //desliga todas as colunas
        BitSet(PORTC,i);      //liga a coluna correspondente
        for(j = 0; j < 3; j++){ //teste para cada bit, atualiza a variável
            if (BitTst(PORTB,j)){
                BitSet(valorNovo,(i*3)+j);
            }else{
                BitClr(valorNovo,(i*3)+j);
            }
        }
    }
}
```

(25 pts Questão 4): Faça um programa que receba comandos enviados pela serial. Quando chegar a string "IDE" pela serial, o programa deve devolver pela serial o número de identificação do programa. Este número é o próprio numero de matricula do projetista do programa. Lembre-se: a função RecebeSerial() retorna valor 0 (zero) quando não há nada a receber na serial; a string "IDE" é composta de 3 caracteres que devem ser recebidos em sequência; não é possível ter certeza de quando um caracter será recebido. Pode-se utilizar as funções do serial.h.

```
#include "basico.h"
#include "config.h"
#include "serial.h"

void main(void) {
    char qt_letras = 0; //conta quantas letras corretas já chegaram
    char letra = 0;
    InicializaSerial();
    for(;;){ //loop infinito
        //aguarda chegar uma letra
        while(letra == 0){
            letra = RecebeSerial();
        }
        //testa se chegou 'n'
        if (letra == 'I'){
            qt_letra=1; //já chegou uma letra certa
        }
        //testa se a letra é 'o' e se já chegou 'n' antes
        if ((letra == 'D') && (qt_letra == 1)){
            qt_letra=2; //já chegaram duas letras certas
        } else {
            qt_letras = 0; //chegou uma errada, tem que reiniciar
        }
        //testa se a letra é 'm' e se já chegou 'n' e 'o' antes
        if ((letra == 'E') && (qt_letra == 2)){
            qt_letra=3; //já chegaram três letras certa
        } else {
            qt_letras = 0; //chegou uma errada tem que reiniciar
        }
    }
}
```

```

//se chegou a palavra "IDE" via serial envia a identificação via serial
if(qt_letras == 3){
    EnviaSerial("1");
    EnviaSerial("1");
    EnviaSerial("9");
    EnviaSerial("5");
    EnviaSerial("2");
    qt_letras = 0;
}
}
}

//Funções disponiveis
//timer.h
    void InicializaTimer(void);
    void ResetaTimer(unsigned int);
    void AguardaTimer(void);
//disp7seg.h
    void InicializaDisplays(void);
    void MudaDigito(char val, char pos);
    void AtualizaDisplay(void);
//adc.h
    void InicializaADC(void);
    unsigned int LeValorAD(void);
//serial.h
    void InicializaSerial(void);
    unsigned char RecebeSerial(void);
    void EnviaSerial(unsigned char);

```